

SoundHound CIE Interview Prep

ROAST Deep Dive

Cognitive Implementation Engineer I

Amelia Global Services, Bengaluru

Generated: June 2026

ROAST — Resume Critic AI (Deep Dive)

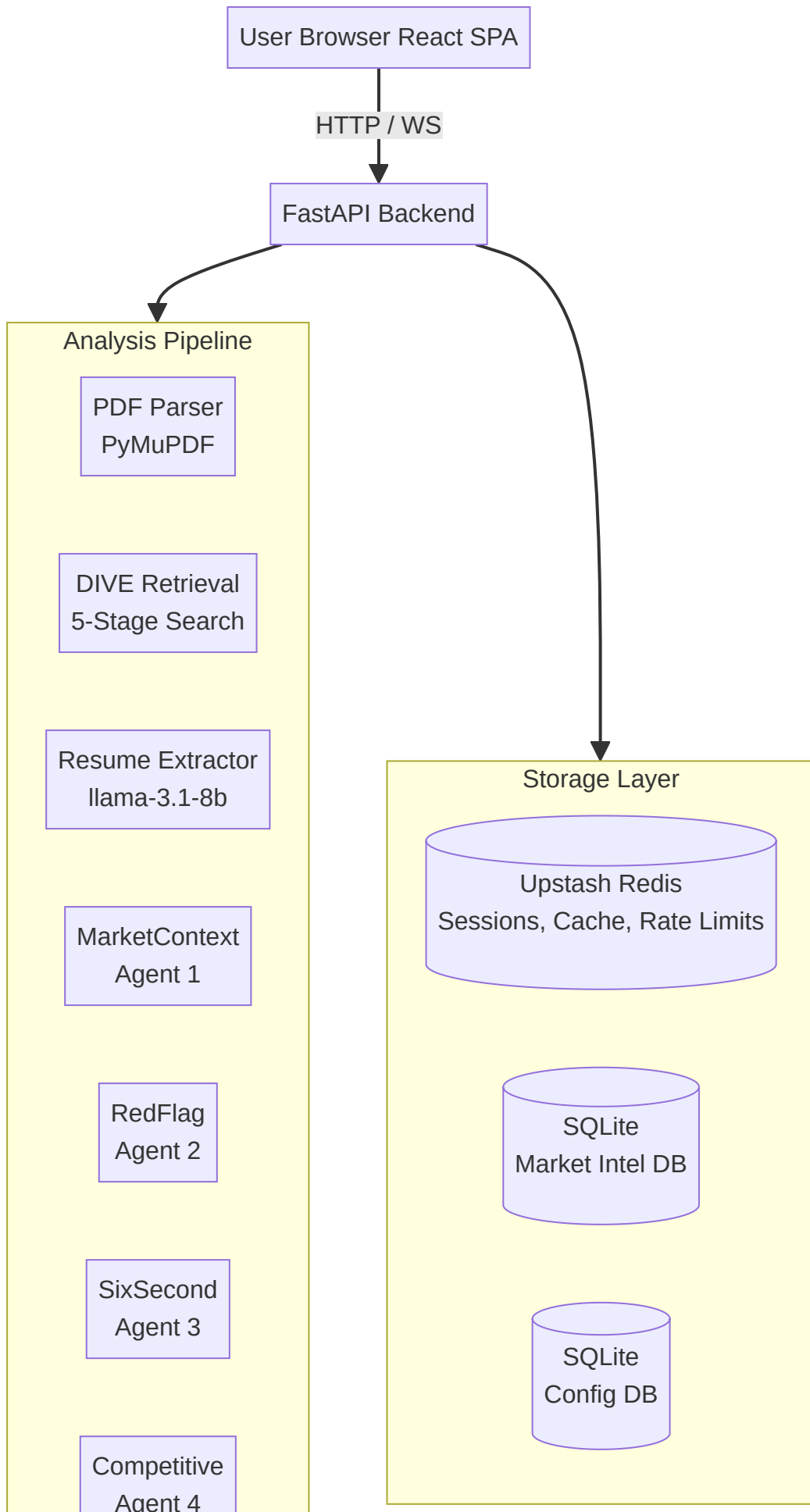
*****For****: Complete refresher. Understand everything you built.*

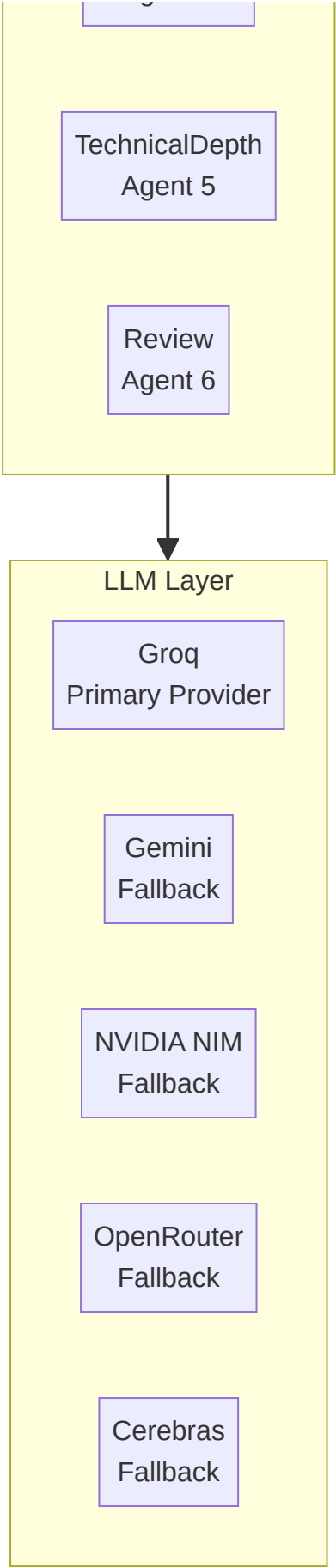
*****Goal****: Answer any interview question about ROAST architecture, agents, pipeline, or tradeoffs.*

What is ROAST?

A web app where users upload a resume → get a market-calibrated AI review. 6 agents analyze the resume in parallel, grounded in real market data. ---

High-Level Architecture





Tech Stack

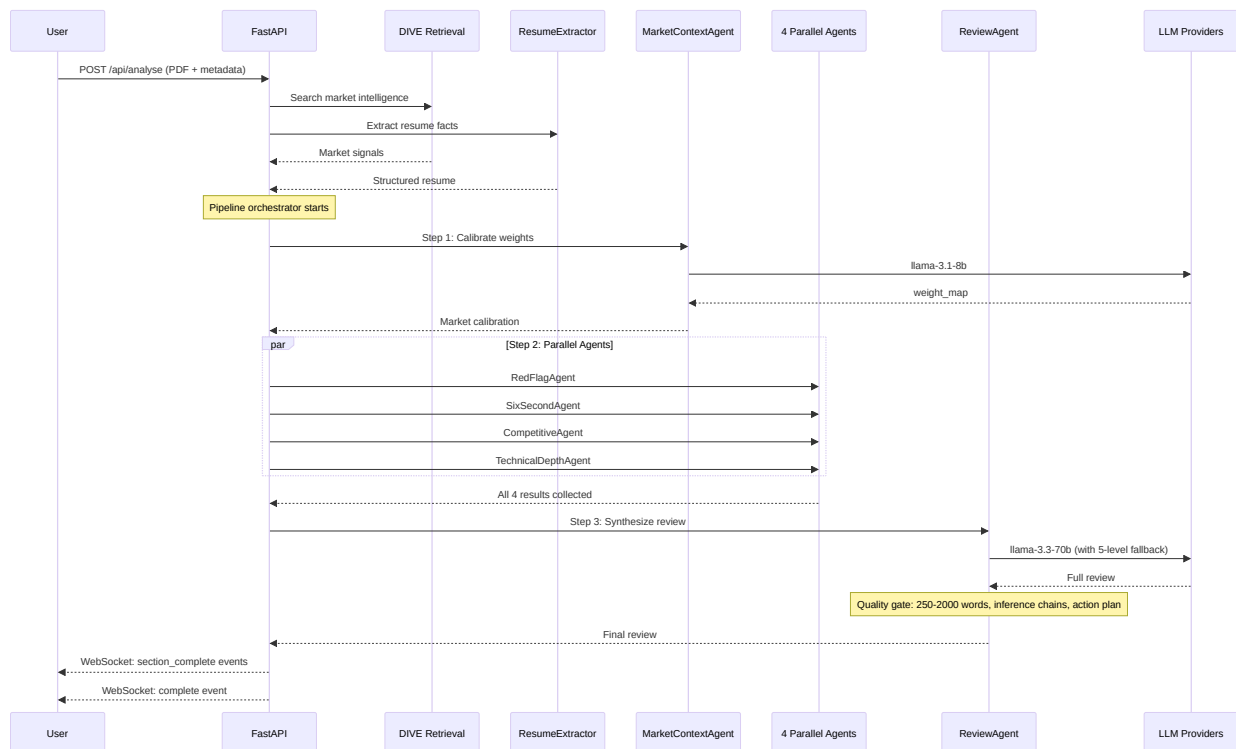
Component	Technology	Why Chosen
Backend	FastAPI	Async, native WebSocket, auto OpenAPI docs
Frontend	React 19 + Vite 8 + Tailwind v4	Modern, fast HMR
LLM Primary	Groq (multi-key)	14,400 RPD for 8B models, free tier
LLM Fallbacks	Gemini, NIM, OpenRouter, Cerebras	5 providers = no single point of failure
Cache/Session	Upstash Redis	Serverless Redis, free tier, survives restarts
Market DB	SQLite (market_intel.db)	Pre-built via monthly ingestion. No external DB needed
Config DB	SQLite (market_config.db)	Company lists, salary bands, role weights
PDF	PyMuPDF (fitz)	Fast, extracts links from annotation layer
Embeddings	Gemini gemini-embedding-001	3072-dim. API-based, no GPU needed
Search	SQLite FTS5 + numpy	BM25 + vector. No external vector DB needed
Observability	Langfuse v4	LLM call tracing, user feedback
Deployment	Docker → Cloud Run	Single container, auto-scaling to 0

Project Structure

```
''' roast/ | — backend/ | | — main.py # FastAPI entry, routers, startup/shutdown | | — config.py # Env
vars: API keys, rate limits, thresholds | | — market_data.py # 982 lines. SQLite config DB CRUD + seed
data | | — pdf_reader.py # Resume PDF → text + link extraction | | — agents/ # 6 AI agents | | —
resume_extractor.py # Pre-pipeline: structured resume facts | | — market_context_agent.py # Agent 1
(runs first): weight calibration | | — red_flag_agent.py # Agent 2: 11 red flag categories | | —
six_second_agent.py # Agent 3: recruiter scan + trajectory | | — competitive_agent.py # Agent 4:
percentile + CTC estimation | | — technical_depth_agent.py # Agent 5: project eval with web search |
| | — review_agent.py # Agent 6 (runs last): full review synthesis | | — followup_agent.py # Post-
review: follow-up Q&A | | — prompts/ # System prompts per agent | | — llm/ # Provider clients +
routing | | — router.py # Routes to providers with fallback chains | | — groq_client.py # Distributed
key rotation via Redis | | — gemini_client.py # Gemini fallback | | — openrouter_client.py #
Emergency fallback | | — nvidia_nim_client.py # NVIDIA NIM fallback | | — cerebras_client.py #
Cerebras fallback | | — langfuse_client.py # Observability wrapper | | — circuit_breaker.py # 3-
state per-provider breaker | | — pipeline/ | | — orchestrator.py # 480 lines. Coordinates ALL agents |
| — retrieval/ | | — dive.py # 645 lines. 5-stage retrieval pipeline | | — routes/ | | — analyse.py #
POST /api/analyse | | — session.py # POST /api/session-init | | — websocket.py # WS
/api/ws/{session_id} | | — followup.py # POST /api/followup | | — cron.py # POST /refresh-market-
intel | | — token_feedback.py # POST /api/token, /api/feedback | | — storage/ | | —
session_store.py # Redis session CRUD | | — rate_limit.py # IP-based, IST midnight reset | | —
```

redis_client.py # Upstash Redis singleton | └─ corpus/ # Anonymized signal DB for calibration |─
 frontend/ # React SPA | └─ src/ | └─ App.jsx # View routing: landing vs analysis | └─ lib/api.js # 8
 API functions + WebSocket creator | └─ hooks/ | | └─ useWebSocket.js # WS with polling fallback +
 heartbeat | | └─ useInferenceToggle.js # localStorage toggle | └─ components/ | └─
 LandingPage.jsx # Upload form (440 lines) | └─ AnalysisProgress.jsx # 8-step progress (201 lines) |
 └─ ResultsPage.jsx # Results orchestrator (229 lines) | └─ TLDRBlock.jsx # Shortlist chance + fixes |
 └─ MarketPulse.jsx # Salary, sentiment, signals | └─ ReviewDocument.jsx # 6-section accordion (266
 lines) | └─ Feedback.jsx # Thumbs + token unlock | └─ SkeletonLoader.jsx | └─ ingestion/ # Monthly
 market intel builder | └─ pipeline.py # Orchestrator. 10 queries/combo | └─ extractor.py # LLM signal
 classification | └─ search.py # SQLite FTS5 CRUD | └─ embeddings.py # Gemini embedding + vector
 search | └─ tavily_client.py # Budget-tracked Tavily search | └─ breaking_signal.py # 24h breaking
 news layer | └─ tests/ # 7 test files | └─ scripts/ | └─ populate.py # Ingestion for all 70 combos |─
 reembed.py # Regenerate missing embeddings ``` ---

Agent Pipeline



The 6 Agents in Detail

Agent 1: MarketContextAgent (runs FIRST)

``` Example output for "SDE at Indian Startup": { "dsa\_weight": 0.30, "projects\_weight": 0.25,  
 "cgpa\_weight": 0.15, "experience\_weight": 0.20, "college\_tier\_weight": 0.10 } ```



## Agent 2: RedFlagAgent (parallel)

1. **Hedge words** ("familiar with," "exposed to") — shallow knowledge
2. **Unverifiable skills** — claims without evidence
3. **Missing contact** — no email/phone/LinkedIn
4. **CGPA consequences** — below role threshold
5. **Buried lead** — impressive fact in boring sentence
6. **Responsibility without outcome** — just listed duties
7. **Date arithmetic** — gaps/overlaps in timeline
8. **Hidden CGPA** — graduation year shown but GPA hidden
9. **Generic filler** — anyone could write this
10. **ATS keyword gaps** — missing terms for target role
11. **Role-specific mistakes** — wrong tech stack claims

## Agent 3: SixSecondAgent (parallel)

## Agent 4: CompetitiveAgent (parallel)

- Percentile range (e.g., "45th–55th percentile")
- Expected CTC range (e.g., "₹12-18 LPA")
- Strengths and weaknesses for THIS specific role

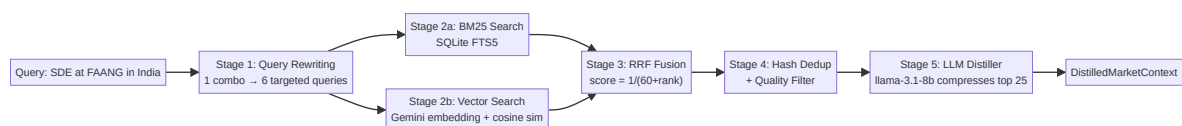
## Agent 5: TechnicalDepthAgent (parallel, 90s timeout)

## Agent 6: ReviewAgent (runs LAST)

1. What's Working
2. What's Hurting You
3. Career Story
4. Competitive Position
5. Action Plan

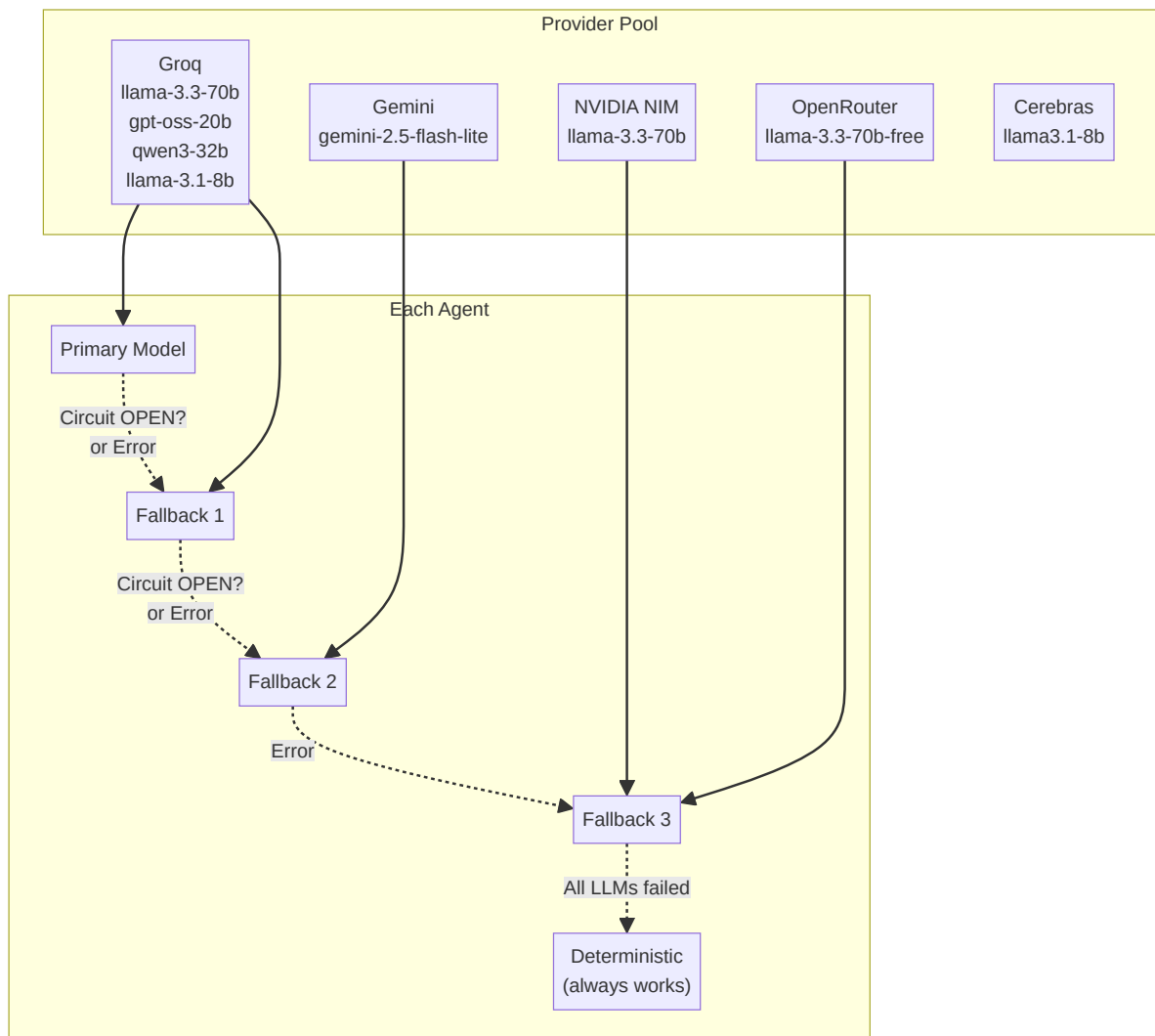
---

## DIVE Retrieval (5 Stages)



---

## LLM Routing & Fallback



- **CLOSED:** Normal operation
- **OPEN (after 3 failures):** Fast-fail for 5 minutes
- **HALF-OPEN (after 5 min):** Allow 1 test call
- **Success → CLOSED. Failure → OPEN again.**

| Agent          | Primary              | Fallback 1   | Fallback 2 | Fallback 3                |
|----------------|----------------------|--------------|------------|---------------------------|
| Review         | llama-3.3-70b (Groq) | gpt-oss-20b  | qwen3-32b  | Gemini → NIM → OpenRouter |
| RedFlag        | llama-3.3-70b (Groq) | llama-3.1-8b | —          | —                         |
| SixSecond      | qwen3-32b (Groq)     | llama-3.1-8b | —          | —                         |
| Competitive    | gpt-oss-20b (Groq)   | NIM          | —          | —                         |
| TechnicalDepth | gpt-oss-120b (Groq)  | llama-3.1-8b | —          | —                         |
| MarketContext  | llama-3.1-8b (Groq)  | —            | —          | —                         |

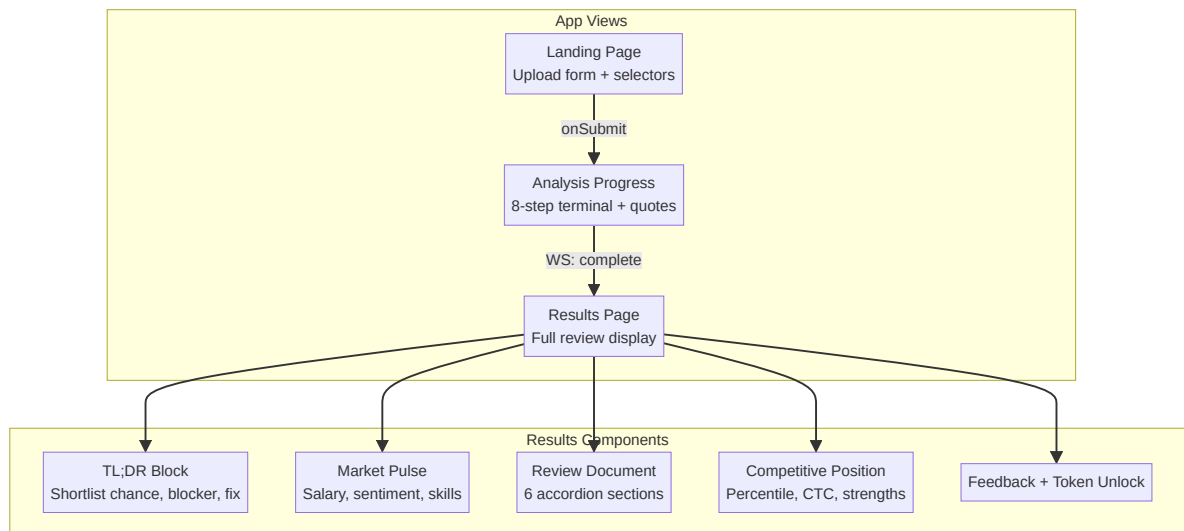
---

## 16 API Endpoints

| Method | Route                   | What It Does                                       |
|--------|-------------------------|----------------------------------------------------|
| POST   | /api/session-init       | Create session (role, company, market, experience) |
| POST   | /api/analyse            | Upload PDF resume (multipart)                      |
| WS     | /api/ws/{session_id}    | Real-time progress + results stream                |
| GET    | /api/session/{id}/state | Poll for reconnection after WS drop                |
| GET    | /api/session/{id}       | Raw session data                                   |
| POST   | /api/followup           | One-click follow-up Q&A                            |
| POST   | /api/feedback           | Useful/not-useful vote                             |
| POST   | /api/token              | Request email token for extra analysis             |
| POST   | /api/token/verify       | Validate token                                     |
| POST   | /refresh-market-intel   | QStash monthly cron (HMAC verified)                |
| GET    | /health                 | Liveness + total analysis count                    |
| GET    | /share/{session_id}     | Public TL;DR preview                               |

---

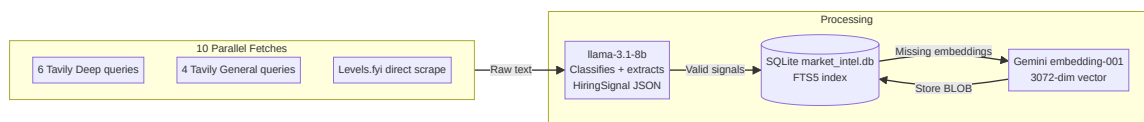
## Frontend Architecture



1. **Landing**: Form with 4 selectors (role, experience, company, market), PDF dropzone (5MB max), optional context/JD/GitHub, consent checkboxes
2. **Analysis** → either Progress (streaming) or Results (complete)
  - Connects to `WS /api/ws/{session_id}`
  - Processes: `ping` → reply `pong`, `section_complete` → accumulate results, `complete` → show results
  - **Polling fallback**: If WS disconnects, poll `GET /api/session/{id}/state` every 5s
  - **Heartbeat**: 3 missed pings (45s) → switch to polling

---

## Ingestion Pipeline (Monthly)



1. Fire 10 search queries (6 deep + 4 general) via `asyncio.gather`
2. Handle truncated results via Jina Reader fallback
3. Classify each result with llama-3.1-8b (is this a job posting? salary survey? blog? discard?)
4. Extract structured HiringSignal: `signal_type`, `skills`, `salary_range`, `sentiment`, `trust_weight`
5. Store in SQLite `market_signals` table with auto-synced FTS5 index
6. Generate Gemini embeddings (3072-dim), store as BLOB

---

## Key Architecture Decisions

1. **SQLite instead of vector DB:** A full Pinecone/Qdrant deployment was overkill. SQLite FTS5 + numpy cosine similarity handles the scale (~10K signals). No external service = zero cost, zero ops.
1. **5-provider fallback:** Free-tier LLM APIs are unreliable. Circuit breakers + fallback chains ensure no single provider failure blocks a user. The deterministic fallback guarantees something useful is always returned.
1. **Agent parallelism:** Agents 2-5 run in parallel (asyncio.gather). This cuts total analysis time from ~3 min to ~1.5 min. Agent 1 must run first (its output is needed by everyone). Agent 6 must run last.
1. **WebSocket + polling dual-mode:** WebSocket for real-time streaming during normal operation. HTTP polling fallback for reconnection. Heartbeat monitor detects silent disconnects.
1. **\$0 infrastructure:** All services on free tiers (Groq, Gemini, Upstash, Tavily, Resend, Cloud Run free quota). This constrains architecture (rate limits, budget tracking) but keeps operating cost at zero.